Laboratório Funcional · Aula 12

Breno Nosima · RA 24113673-2 · Turma A
Gerado em 16/10/2026, 20:01:28 · tempo registrado no cronômetro: 25:52

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A função agora() é impura porque depende do horário atual. A função registra() é impura porque utiliza print(), causando um efeito colateral.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque a função altera uma variável global a cada chamada. Isso dificulta os testes, pois o resultado depende do estado anterior.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = 3 × 4 + 2 = 14. Já compor(g, f)(4) = (4 + 2) × 3 = 18. A ordem das funções altera o resultado.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted() cria uma nova lista ordenada, enquanto sort() modifica a lista original. sorted() segue melhor o estilo funcional porque não altera os dados originais.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
Original:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

Sem o apóstrofo: erro, pois A não está definida como função.
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo impede que a lista seja executada como código. Sem ele, o Scheme tenta chamar A como uma função.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme funciona por causa da otimização da recursão em cauda. No Python ocorre erro de recursão.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, a pilha de chamadas cresce porque existem operações pendentes. Na recursão em cauda, a chamada recursiva é a última operação e pode reutilizar o espaço.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza chamadas em cauda, evitando o crescimento da pilha. O Python não realiza essa otimização e possui um limite de recursão.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis) 0 (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis) 0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi montar a recursão. Usei null? para verificar se a lista estava vazia, car para acessar o primeiro elemento e cdr para continuar com o restante.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

[0,1,4,9,16]

42

Saídas (original e com member 17)

```
Original:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Com member 17 squares: a busca não termina.
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell utiliza avaliação preguiçosa, calculando apenas os elementos necessários. A divisão por zero não gera erro porque o segundo argumento não é utilizado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O número 17 não existe na lista de quadrados, então a busca continua indefinidamente. O número 16 é encontrado. Para corrigir, a busca deve parar quando o elemento atual ultrapassar o valor procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15

[5, 10, 15]

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Porque a função utiliza um fechamento (closure), que mantém o valor de n armazenado mesmo depois que somador() termina.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose() executa primeiro g e depois f, resultando em 14. andThen() executa primeiro f e depois g, resultando em 18. compose() corresponde ao compor(f, g) da prática 1.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 }
];
const aprovados = alunos
  .filter(a ⇒ a.nota ⩾ 6)
  .map(a ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Exercício 1 (Java): 244
Exercício 2 (JavaScript): ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Achei a versão funcional mais fácil de ler porque o código fica menor e mais organizado. Já a versão com laço pode ser mais fácil de depurar, pois permite acompanhar cada passo.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão funcional é mais fácil de entender porque utiliza menos código e mostra diretamente o cálculo. Aceitaria um custo maior de execução em sistemas administrativos simples, mas não em sistemas de tempo real que precisam de respostas rápidas.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi descobrir que é possível trabalhar com listas infinitas e calcular apenas os valores necessários.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está em b <- h, pois h é um elemento, não uma lista. O compilador infere que h deveria ser uma lista por causa do operador <-, causando incompatibilidade de tipos. A correção é:

```
sort [] = []
sort (h:t) = sort [b | b <- t, b <= h]
        ++ [h] ++
        sort [b | b <- t, b > h]
```